

Detecting Dark Patterns in UI Screenshots

[Jiangbo Deng, Kairui Feng, Yixun Hong, Zhiyu Wang, Shuhuai Zhang]

June 9, 2025

1 Summary

Our project presents a comprehensive approach for the automated detection of dark patterns in User Interface (UI) screenshots using Vision-Language Models (VLMs). To address the limitations of traditional manual or text-based analysis, we first constructed a custom dataset by integrating and refining public resources, namely AidUI and UI-Guard. This resulting dataset focuses on key dark pattern categories and is structured in the COCO JSON format to facilitate detection and localization tasks.

At the core of our methodology, we employed QLoRA (Quantized Low-Rank Adaptation), a Parameter-Efficient Fine-Tuning (PEFT) technique, to instruction-tune the Qwen2-VL and Qwen2.5-VL models. This approach enabled the models to efficiently learn to analyze UI screenshots and generate structured JSON outputs detailing the type, description, and bounding box coordinates of detected dark patterns.

To thoroughly assess model performance, we established a multi-level evaluation framework encompassing presence detection, type classification, and instance localization. Experimental results clearly demonstrate that the performance of models fine-tuned with QLoRA significantly surpasses their base versions. Notably, the fine-tuned Qwen2.5-VL model emerged as the top performer across all evaluation tiers, achieving high metrics in accurately identifying and localizing dark patterns.

In conclusion, our study successfully validates that instruction-tuning VLMs with QLoRA is an effective approach for automated visual dark pattern detection. Our work provides a robust methodology and a crucial performance benchmark for future efforts to identify and mitigate deceptive designs in digital interfaces.

2 Introduction

2.1 The definition of Dark Patterns

User interaction with digital platforms, including websites and mobile applications, is a ubiquitous and integral part of modern life. While engaging in actions such as browsing, purchasing, or subscribing, users expect intuitive and transparent interfaces. However, these interactions can

sometimes lead to unexpected and potentially detrimental outcomes, such as unintentional subscriptions or significant difficulty in locating opt-out mechanisms. These usability issues are often not incidental design flaws but rather result from deliberate design choices aimed at influencing user behavior in ways that benefit the service provider.

These intentionally deceptive design elements are collectively known as 'dark patterns'. Formally, a dark pattern is defined as a user interface or user experience design element specifically engineered to mislead or manipulate users into taking actions that are against their best interests and often unintended. Understanding dark patterns requires recognizing their key characteristics:

- **Intentionality:** Dark patterns are not accidental errors; they are consciously designed and strategically implemented to achieve specific organizational objectives, frequently prioritizing business outcomes over user welfare and autonomy.
- **Manipulation:** These designs leverage psychological principles, cognitive biases (e.g., default bias, framing effects), user inattention, or established behavioral patterns to guide users toward predetermined actions, often diverging from the user's conscious intent or preferences.
- **Detriment:** The deployment of dark patterns typically results in outcomes disadvantageous to the user. This can manifest as financial loss (e.g., unintended purchases, unwanted subscriptions), wasted time, or the compelled disclosure of personal data, while simultaneously yielding benefits for the entity employing the pattern (e.g., increased revenue, data collection, user lock-in).

The prevalence and potential harm of dark patterns necessitate effective identification and mitigation strategies. This paper aims to investigate the nature of dark patterns in user interfaces and explore potential methodologies for their automated detection, thereby contributing to the development of more ethical and user-centric digital environments.



Figure 1: Disguised Ads

2.2 Types of dark patterns

From the website <https://www.deceptive.design/types>. There are multiple types for dark patterns.

- **Comparison prevention:** Deliberately hinders effective product comparison via complex, hidden, or inconsistent information. Frustrates users, leading to rushed, biased decisions prioritizing provider profit over user interest.
- **Confirmshaming:** Uses emotional manipulation (guilt/shame) via belittling opt-out language. Pressures users into compliance, exploiting vulnerability for provider gain over user autonomy and well-being.
- **Disguised Ads:** Deliberately blurs advertising and native content, mimicking elements to confuse users into clicking ads. Aims to boost ad revenue/clicks by deception, prioritizing provider benefit over transparent user experience. 1
- **Fake Scarcity:** Creates false impressions of limited supply or high demand using misleading counts or figures. Triggers scarcity bias, pressuring rushed decisions driven by fear of missing out, prioritizing provider conversions.
- **Fake Social Proof:** Uses fabricated endorsements (false reviews/activity) to create misleading popularity/credibility. Exploits social proof bias, tricking users into conforming or trusting based on false evidence for provider gain.
- **Fake Urgency:** Applies false time pressure (fake timers) to induce anxiety and hinder critical evaluation. Pushes users into rushed actions against their interest, prioritizing provider conversions by exploiting vulnerability.
- **Forced Action:** Requires users perform an undesirable task to achieve a desired one. Often bundles actions (like consents), making refusal difficult. Leverages user motivation to extract unwanted actions/-data, prioritizing provider gain.
- **Hard to Cancel:** Makes subscribing easy but terminating a service difficult. Cancellation options hidden or complex, frustrating users into continued payment for unwanted services. Prioritizes revenue over user autonomy.
- **Hidden Costs:** Unadvertised fees revealed late in transaction (checkout). Users, having invested time/effort, accept the higher price. Exploits sunk cost bias, prioritizing increased revenue via unexpected final costs.
- **Hidden Subscription:** Enrolls users into recurring payments without clear consent, via sneaking or misdirection. Users unknowingly subscribed during seemingly one-time purchase. Minimizes notifications to maximize revenue.
- **Nagging:** Persistently interrupts users with requests, disrupting actions. Repetitive prompts deplete attention, wearing down resistance. Aims for compliance against user interest to end interruption, prioritizing provider goals.
- **Obstruction:** Deliberately creates barriers hindering task completion or information access. Exhausts users hoping they abandon goals contrary to business objectives, prioritizing provider control over user autonomy.
- **Preselection:** Leverages the default effect by presenting pre-chosen options (pre-ticked boxes, items in cart). Exploits inertia and inattention to influence decisions, subtly guiding users towards provider-preferred choices.2
- **Sneaking:** Intentionally conceals or delays relevant information (hidden costs, unwanted consequences). Draws users into transactions under false pretenses, manipulating into unintended actions for provider gain.

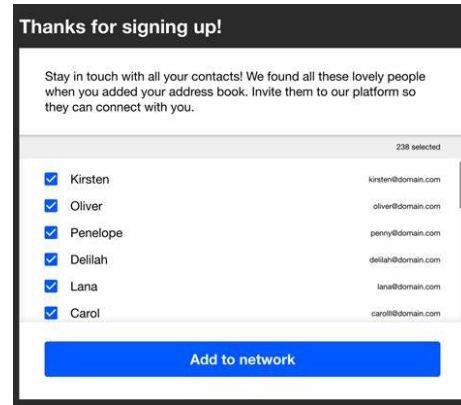


Figure 2: Preselection

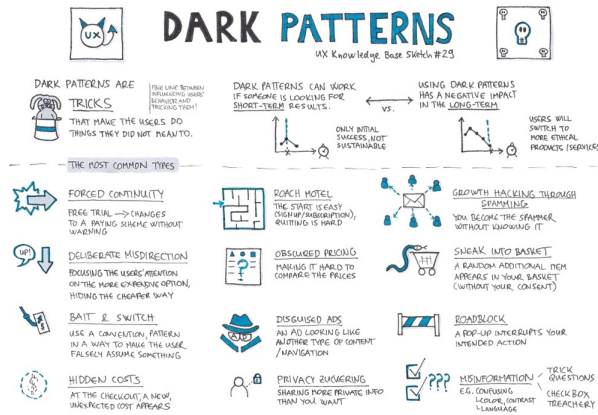


Figure 3: Types of dark patterns

2.3 Harm of dark pattern

The pervasive use of dark patterns in user interfaces creates substantial negative consequences, impacting both users and the broader digital landscape. For users, these deceptive designs directly lead to tangible harm, including unintended financial loss, compromised personal privacy, and significant wasted time and mental effort spent navigating manipulative interfaces. Beyond individual detriment, the prevalence of dark patterns erodes user trust, fosters an environment of unfair competition among businesses, and increasingly subjects organizations to legal challenges and regulatory penalties. Given the intentional nature and scale at which these patterns are deployed across numerous websites and applications, manual identification efforts are inherently inefficient and insufficient. Effectively combating this widespread issue and promoting ethical design practices therefore necessitates the development of robust and scalable solutions. This underscores the critical need for designing and implementing an automated workflow specifically capable of detecting and identifying the presence of dark patterns within user interfaces and experiences.

3 Motivation and Background

The pervasive use of dark patterns in user interfaces presents a significant challenge in the digital landscape. These intentionally deceptive design elements directly harm users through unintended financial loss, privacy risks, and wasted time. Furthermore, they negatively impact the industry by creating unfair competition and exposing businesses to legal and regulatory risks. Effectively identifying and mitigating these patterns is crucial for fostering ethical design and protecting users. Given the widespread nature and increasing sophistication of dark patterns, manual detection methods are inherently in-

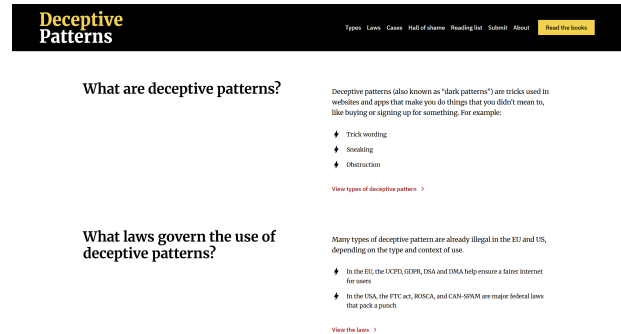


Figure 4: darkpatterns.org

efficient and unscalable, necessitating the development of automated approaches.

Understanding this problem requires examining the existing research landscape. The concept of dark patterns was pioneered by Harry Brignull, who cataloged and defined various types on darkpatterns.org. Subsequent research, such as that by Gray et al., refined classification schemes including 'nagging,' 'obstruction,' and 'sneaking,' providing a structured understanding of these patterns. Large-scale studies on shopping websites and mobile applications have further substantiated the prevalence and impact of dark patterns.

Existing detection methods can be broadly categorized. Initial efforts relied on manual reporting by researchers or users, which offered high accuracy but was extremely time-consuming and lacked scalability for evolving applications. Semi-automated methods, such as clustering UI patterns to simplify manual review, improved efficiency for known patterns but still involved significant manual effort and struggled with novel designs. Text-based classification techniques have also been explored, leveraging UI text to identify certain patterns automatically. However, these methods are limited to textual content, prone to context-dependent errors, require extensive labeled data, and have a constrained scope of detectable patterns.

Many dark patterns are inherently visual, relying on layout, color, and element placement rather than just text. The limitations of existing detection methods, particularly their reliance on manual analysis, textual data, or restricted scope, highlight a critical gap in automated visual detection from UI representations. Therefore, this paper is motivated by the necessity to design an automated workflow capable of effectively detecting a wide range of dark patterns directly from UI screenshots, addressing the scalability and visual analysis challenges faced by previous approaches.

4 Highlevel WorkFlow

The workflow for our enhanced Dark Pattern identification is illustrated in the Figure 5.

As depicted, we initially obtained datasets from AidUI and UIGuard, which underwent preliminary cleaning, processing, and merging. Subsequently, considering the quantity and quality factors of various data types, we reconstructed the dataset according to the classification standards in AidUI, resulting in dataset_final. This dataset was then partitioned into training, evaluation, and test sets at a 7:1:2 ratio, respectively.

Utilizing the training set, we employed a QLoRA-based instruction fine-tuning method to enhance the Dark Pattern classification capabilities of the Qwen2-VL-7B-Instruct and Qwen2.5-VL-7B-Instruct models. Performance evaluation was then conducted on the test set.

Finally, we organized and analyzed the results, and visually presented our model’s performance and remaining challenges.

5 Technique Details

Large pre-trained models, particularly Vision-Language Models (VLMs), have demonstrated exceptional capabilities in content understanding and generation. However, efficiently applying these general-purpose models to specific downstream tasks, such as "dark patterns" identification, which have high requirements, still presents significant challenges. These challenges arise not only from the models’ large parameter counts and corresponding computational resource demands but also from the specific nature of the tasks and the need for a nuanced understanding by the models.

In relevant project practices, the adopted technical approach evolved from initial attempts to directly guide the model using Prompt Engineering to the subsequent stage of employing QLoRA, a Parameter-Efficient Fine-Tuning (PEFT) technique, for Instruction Tuning. This evolution clearly reflects a strategic shift in handling complex visual tasks: from relying on the model’s inherent capabilities to leveraging data-driven learning for better adaptation to specific tasks. Simultaneously, this transformation also highlights the urgent need for efficient model adaptation methodologies when facing dual pressures of resource constraints (e.g., limited hardware capabilities) and performance metric requirements.

5.1 Parameter-Efficient Fine-Tuning

5.1.1 Definition, Objectives, and Advantages

Parameter-Efficient Fine-Tuning (PEFT) refers to a series of technical methods designed to adapt large pre-trained

models with minimized computational and storage overhead. Its core idea is to fine-tune only a small subset of the model’s parameters, rather than adjusting all of them, thereby significantly reducing resource consumption while striving to maintain performance levels comparable to full parameter fine-tuning.

PEFT’s advantages are particularly prominent in scenarios involving device limitations and complex tasks. These primarily include: first, a significant reduction in GPU memory and training time required during fine-tuning; second, by freezing most of the pre-trained model’s parameters, PEFT helps mitigate catastrophic forgetting, thereby preserving the general knowledge acquired during the pre-training phase; finally, PEFT greatly enhances deployment flexibility by allowing the storage of a small, independent PEFT module for each task, rather than a copy of the entire model parameters. QLoRA, as a PEFT method, has been successfully applied in the instruction tuning process within the project.

5.1.2 Method Classification

PEFT encompasses various specific methods, which can be broadly categorized into additive methods (e.g., Adapters, Soft Prompts/Prompt Tuning), selective methods, and reparameterization methods (e.g., LoRA). Among these, LoRA and its quantized version, QLoRA, fall under the category of reparameterization PEFT, which achieves parameter-efficient adaptation by introducing and training low-rank matrices.

5.2 Prompting Strategies

5.2.1 Prompt Engineering

Prompt Engineering is a technique that guides pre-trained models (especially Large Language Models (LLMs) and Vision-Language Models (VLMs)) to perform specific tasks or generate outputs in a particular format by carefully designing the input text (i.e., the "prompt"), without updating the model’s weights. This method aims to leverage the model’s existing knowledge and instruction-following abilities acquired during the pre-training phase.

Its core mechanisms primarily include: providing explicit instructions, which involves giving clear and specific task descriptions, execution steps, or desired behaviors to the model; utilizing few-shot learning by including a small number of task-relevant input-output examples within the prompt to guide the model in understanding the inherent patterns of the task; and formatting the output, explicitly specifying the structure of the output results (e.g., JSON, Markdown) to ensure the usability and standardization of the model’s output.

Our prompt clearly defines the model’s role (UI/UX review expert), the task (identifying dark pattern types),

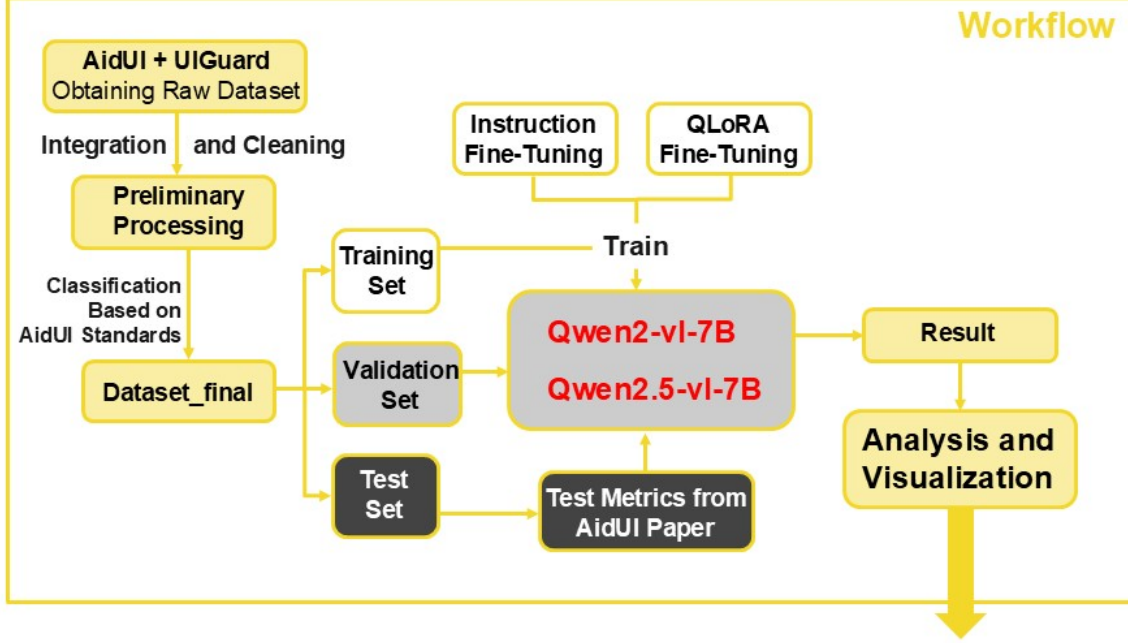


Figure 5: The workflow for our enhanced Dark Pattern identification

specific dark pattern classifications, detailed JSON output format specifications, and conditional output logic (presence vs. absence of dark patterns).

Although for capable large models, well-designed prompt engineering can quickly achieve zero-shot or few-shot adaptation to new tasks and effectively control output formats without retraining the model, its inherent limitations should not be overlooked. For instance, for tasks requiring deep reasoning, nuanced understanding, or knowledge beyond the model’s pre-training scope (e.g., certain complex dark pattern identification scenarios), relying solely on prompt engineering may not yield ideal results. The model might not be able to generalize to all unseen complex situations based on prompts alone, which aligns with the "poor performance" observed in the initial stages of the project. Furthermore, model performance is highly dependent on the phrasing and structure of the prompt, and minor variations can lead to significant differences in the final outcome. Most importantly, prompt engineering primarily relies on the model’s existing capabilities and struggles to enable the model to learn the deeper patterns and implicit knowledge specific to new tasks from new data.

When prompt engineering proves insufficient to meet task requirements, instruction tuning offers a paradigm that allows the model to learn from data by adjusting its parameters.

5.3 Instruction Tuning

In the project, given that prompt engineering did not yield expected results, the technical path shifted towards "partially fine-tuning the model using samples," which represents the practical application of instruction tuning.

5.3.1 Basic Concepts

Instruction tuning is a fine-tuning paradigm aimed at improving the model’s ability to follow natural language instructions and optimizing its behavior for specific tasks or domains. Its core objective is to enable the model to gain a deeper understanding of specific task requirements and generate responses that meet predefined criteria by learning from a large number of (instruction, input, desired output) triplet samples.

Unlike the pre-training phase, which focuses on acquiring general language understanding and generation capabilities, instruction tuning is more focused on "teaching" the model how to perform corresponding operations in response to specific instructions and inputs. For example, in the specific project case, this refers to how to accurately locate and describe dark pattern features based on UI screenshots and the "identify dark patterns" instruction.

5.4 Mechanism Elaboration

Instruction tuning typically employs a supervised learning methodology, training on datasets structured with instructions, inputs (or contextual information), and outputs (or desired responses). Here, "instruction" refers to the natural language text describing the task (e.g., "Identify forced action dark patterns in the image"); "input" is the context-related information (e.g., a UI screenshot); and "output" is the expected result that the model should generate (e.g., a JSON object containing dark pattern type, description, and bounding box coordinates). During training, the model's parameters (or a subset thereof, when using PEFT techniques) are adjusted with the goal of minimizing the discrepancy between the model's actual output and the desired output. Through this process, the model learns to map specific instructions and inputs to correct outputs.

5.5 Visual Instruction Tuning

Visual instruction tuning specifically refers to applying the instruction tuning paradigm to tasks involving visual information, typically for Vision-Language Models (VLMs). Such models need the ability to understand both visual input (like UI screenshots) and textual instructions simultaneously, and to generate responses that integrate information from both modalities.

The "dark patterns" identification task—that is, identifying dark patterns from UI screenshots and outputting results in JSON format—is a typical application scenario for visual instruction tuning. In this scenario, the model must possess the following capabilities: first, visual content understanding, meaning parsing the UI elements present in the screenshot and their spatial layout; second, textual instruction understanding, clearly knowing to search for specific types of dark patterns and strictly adhering to predefined output format requirements; and finally, multi-modal reasoning, integrating visual information with textual instructions to determine the presence of dark patterns and to locate and describe them.

5.6 Efficacy Analysis and Challenges

Compared to pure prompt engineering, instruction tuning enables the model to learn from a large amount of task-relevant data, thereby exhibiting higher accuracy and robustness on specific tasks. Furthermore, it significantly enhances the model's instruction-following capabilities, allowing it to understand and execute complex or subtle instructions more effectively. After training with sufficiently diverse instruction data, the model may also gain the ability to generalize to new, unseen but similar instructions or scenarios.

However, instruction tuning also faces significant challenges. Firstly, the success of instruction tuning heavily relies on large-scale, high-quality, and diverse instruction datasets, and constructing such datasets, especially for multimodal tasks, is often costly. Secondly, traditional full-parameter instruction tuning demands substantial computational resources. This is precisely where PEFT methods (such as QLoRA) play a crucial role, making instruction tuning possible under limited resource conditions.

The project adopted the technical path of "partially fine-tuning the model using samples" to address the complexity of the task using the instruction tuning paradigm and effectively manage resource consumption through QLoRA technology.

5.7 LoRA and QLoRA

To achieve efficient instruction tuning, the project employed the QLoRA method from PEFT techniques.

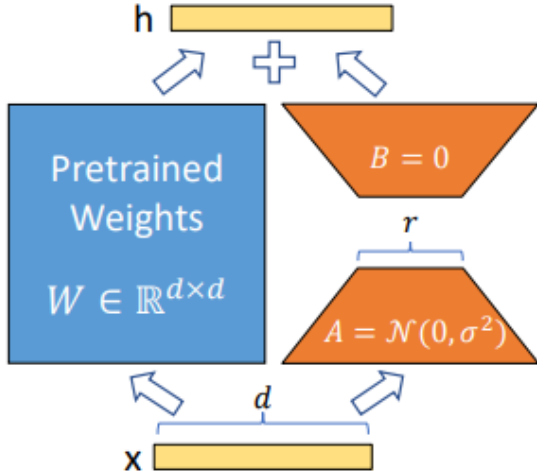
5.7.1 Low-Rank Adaptation

As shown in Figure 6a, the core idea of LoRA is that when large pre-trained models adapt to downstream tasks, the change in their weight matrices (ΔW) typically possesses a low "intrinsic rank." Based on this observation, LoRA does not directly update the original pre-trained weight matrix W_0 ; instead, it introduces two smaller, trainable low-rank matrices A and B alongside it, such that $\Delta W = BA$. During fine-tuning, W_0 remains frozen, and only the parameters of matrices A and B are updated. This method significantly reduces the number of trainable parameters, thereby decreasing memory footprint and computational requirements. Concurrently, during inference, the learned low-rank matrix product BA can be merged with the original weights W_0 , thus introducing no additional inference latency.

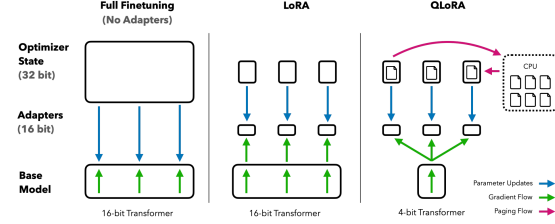
5.7.2 Quantized Low-Rank Adaptation

QLoRA, as a further optimized version of LoRA, is designed to maximize the reduction in memory consumption required for fine-tuning large models, even enabling them to run on a single consumer-grade GPU.

As shown in Figure 6b, the core mechanism of QLoRA involves several aspects. Firstly, it **quantizes** the frozen pre-trained base model weights from standard 16-bit or 32-bit floating-point precision to **extremely low precision (typically 4-bit)**, using data types like NF4 (NormalFloat4). This dramatically compresses the memory occupied by the base model. Secondly, QLoRA continues to use LoRA's adapter fine-tuning approach, training



(a) LoRA presentation, only train A and B.



(b) QLoRA presentation, QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

Figure 6: LoRA & QLoRA presentation.

only the low-rank adapter matrices A and B, which typically maintain a higher computational precision (e.g., 16-bit BFloat16). During gradient computation and forward propagation, the 4-bit base model weights are dynamically **dequantized** back to the required computational precision (e.g., BFloat16) for operations with the LoRA adapters. Subsequently, gradients are backpropagated to the LoRA adapters for parameter updates. Additionally, QLoRA incorporates other optimization techniques, including Double Quantization (DQ) to further compress the space occupied by quantization constants, and Paged Optimizers to effectively manage memory spikes during training.

QLoRA technology allows the scale of models that can be fine-tuned on a single GPU to increase severalfold, while striving to maintain performance levels comparable to 16-bit full-parameter fine-tuning or standard LoRA fine-tuning. This characteristic enables effective instruction tuning of large Vision-Language Models even under "device-constrained" conditions. It is important to note that in QLoRA’s setup, only the parameters of the LoRA adapters (i.e., matrices A and B) are trainable; the base model is quantized and remains frozen.

The introduction of QLoRA technology made it possible to adopt the relatively high computational cost instruction tuning paradigm in the project.

6 Dataset and Metrics

A high-quality, representative dataset and a comprehensive suite of evaluation metrics are indispensable foundations for achieving our goal. This chapter details the construction process of our dataset and the evaluation metric

system adopted.

6.1 Source Datasets Analysis

The construction of our dataset commenced with an analysis of two prominent public datasets in dark pattern research: the CONTEXTDP dataset from the AidUI project and data annotated by the UIGuard project based on the RICO dataset.

6.1.1 AidUI (CONTEXTDP Dataset)

The AidUI project’s CONTEXTDP dataset is a significant resource for DP research, containing 501 mobile and web UI screenshots, with 301 DP instances and 243 Non-DP instances. The mobile UIs were partly sourced from Geronimo et al., expanded to 2994 screenshots from 68 apps, while web UIs were based on Mathur et al.’s work, collecting over 1500 screenshots. CONTEXTDP features a detailed, hierarchical DP taxonomy, as illustrated in Figure 7. Each DP instance is annotated with its type and a bounding box. A notable challenge observed was the limited sample size for certain critical categories, such as “Forced Action” (11 samples) and “Social Proof” (10 samples), which could impede robust model training.

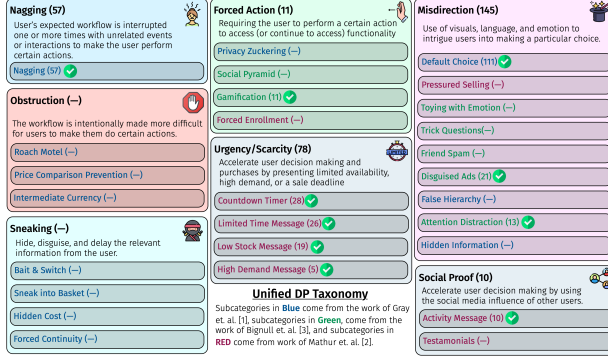


Figure 7: AidUI (CONTEXTDP) Dark Pattern Taxonomy and selected sample counts.

6.1.2 UIGuard Dataset

The UIGuard project provides DP annotations on a subset of the large-scale RICO mobile UI dataset. Its taxonomy also covers common DP types like “Nagging” and “Forced Actions.” UIGuard offers a more substantial number of samples in certain overlapping categories, for instance, “Pop up Ad” (a sub-type of “Nagging”) has 178 samples, and “Privacy Zuckering” (a sub-type of “Forced Actions”) has 117 samples. This dataset served as a valuable resource for augmenting categories underrepresented in CONTEXTDP.

6.2 Final Dataset Construction and Composition

To address the limitations of individual datasets and to create a more robust training corpus, we implemented an integration strategy.

Data Integration and Simplification We supplemented the “Forced Action” and “Nagging” categories from AidUI (CONTEXTDP) with corresponding data from the UIGuard dataset. Concurrently, to manage task complexity and facilitate more effective learning, we consolidated the fine-grained sub-categories into five major DP categories, plus a “NO DP” category. This focused approach aims to improve the model’s ability to distinguish core dark pattern types.

Annotation Format We standardized all data into the COCO JSON format. This format includes distinct sections for images (image metadata like filename, width, height), categories (DP type definitions and IDs), annotations (specifics of each DP instance, including its image_id, category_id, and bounding_box), and info (dataset metadata). The model is trained to generate a textual JSON output conforming to this structure.

```
{
  "info": {
    "info": "info",
    "licenses": [ "license" ],
    "images": [ "image" ],
    "annotations": [ "annotation" ],
    "categories": [ "category" ]
  }
}
```

Figure 8: COCO JSON Format for Dark Pattern Detection. This format includes sections for images, categories, annotations, and dataset info.

Final Dataset Statistics After integration and refinement, our final dataset comprises the following distribution of images per category:

Table 1: Distribution in Final Dataset.

Dark Pattern Category	Number of Images
NO DP	243
Nagging	245
Forced Action	247
Urgency/Scarcity	78
Misdirection	145

This composition provides a more substantial and relatively balanced dataset for model fine-tuning.

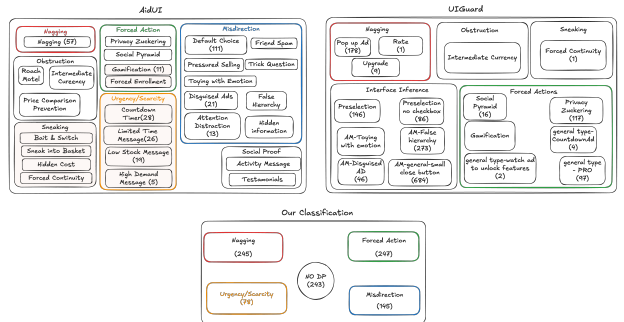


Figure 9: Final Dataset Composition.

6.3 Evaluation Metrics

To comprehensively assess the performance of our fine-tuned models, we adopted a multi-faceted evaluation approach inspired by the AidUI methodology. This involves fundamental metrics applied across three distinct evaluation tiers.

6.3.1 Fundamental Metrics

The core metrics employed are Precision, Recall, F1-score, and Intersection over Union (IoU), as defined below.

- **Precision (P):** The ability of the classifier to correctly identify positive instances among those it predicted as positive. $P = \frac{TP}{TP+FP}$.
- **Recall (R):** The ability of the classifier to find all actual positive instances. $R = \frac{TP}{TP+FN}$.
- **F1-score:** The harmonic mean of Precision and Recall, providing a balanced measure. $F1 = \frac{2 \times P \times R}{P+R}$.
- **IoU (Intersection over Union):** Measures the overlap between the predicted bounding box (B_p) and the ground truth bounding box (B_{gt}). $IoU = \frac{Area(B_p \cap B_{gt})}{Area(B_p \cup B_{gt})}$.

6.3.2 Multi-Level Evaluation Framework

We defined three tiers of evaluation to assess different aspects of model capability.

1) **DP Presence Detection (Image-Level Binary Classification):** This evaluates the model’s ability to determine if a UI screenshot contains any dark pattern.

- *TP (True Positive):* An image containing at least one DP is correctly classified as having a DP.
- *FP (False Positive):* An image with no DPs is incorrectly classified as having a DP.
- *TN (True Negative):* An image with no DPs is correctly classified as having no DPs.
- *FN (False Negative):* An image containing at least one DP is incorrectly classified as having no DPs.

Metrics calculated include overall accuracy, and precision, recall, and F1-score for the "DP present" class.

2) **DP Type Classification (Image-Level, No Bbox):** This assesses the model’s ability to identify the specific types of DPs present in an image, irrespective of their precise location. For each defined DP category (e.g., Nagging, Forced Action):

- *TP (Type):* The image contains a specific DP type, and the model correctly predicts this type for the image.
- *FP (Type):* The image does not contain a specific DP type, but the model predicts it.
- *FN (Type):* The image contains a specific DP type, but the model fails to predict it.

Per-category Precision, Recall, and F1-score are calculated, followed by their Macro-Averages to gauge overall balanced performance across types, aligning with AidUI’s type classification evaluation.

3) **DP Instance Localization (Type & IoU):** This is the most stringent evaluation, requiring correct type classification and accurate localization.

- *TP (Strict):* A predicted DP instance matches a ground truth instance of the same type, and their bounding box IoU meets or exceeds a predefined threshold (e.g., 0.5). The ground truth instance must not have been previously matched.
- *FP (Strict):* A predicted DP instance that is not a TP (due to incorrect type, insufficient IoU, localization in a non-DP area, or duplicate detection of an already matched GT).
- *FN (Strict):* A ground truth DP instance that was not detected as a TP.

Metrics include overall Precision, Recall, and F1-score for these strict instance detections, as well as the Average IoU of the strict TPs. This corresponds to AidUI’s localization performance assessment.

This structured evaluation approach allows for a nuanced understanding of the model’s capabilities, from high-level presence detection to fine-grained instance localization and classification.

7 Model Training Methodology

This section details the approach taken for fine-tuning advanced multimodal large language models for our task of dark pattern detection.

7.1 Base Models

We experimented with two versions of Qwen-VL series: **Qwen2-VL-7B:** A 7-billion parameter vision-language model.

Qwen2.5-VL-7B: An updated iteration, reported to have enhanced capabilities in various vision and language tasks.

These models were selected for their inherent ability to process and understand combined image and text inputs, and to generate coherent textual outputs, which is well-suited for our task of identifying dark patterns from UI screenshots based on textual prompts.

7.2 PEFT

Given the significant computational resources required for full fine-tuning of large models, we employed Parameter-Efficient Fine-tuning (PEFT) techniques, primarily focusing on QLoRA.

7.2.1 QLoRA (Quantized Low-Rank Adaptation)

QLoRA enables efficient fine-tuning by:

- Quantizing the pre-trained model weights to 4-bit using the NormalFloat (NF4) data type.
- Employing Double Quantization to further reduce the memory footprint of quantization constants.
- Utilizing Paged Optimizers to manage optimizer states effectively between GPU and CPU memory.
- Introducing small, trainable Low-Rank Adapters (LoRA) to specific layers of the model (typically attention and feed-forward network linear layers), while keeping the base model weights frozen.

In our implementation, QLoRA was configured with 4-bit loading, NF4 quantization, and double quantization. LoRA parameters such as rank ($r = 32$), alpha ($\alpha = 64$), and dropout (0.05) were applied to target modules including `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. The language model head (`lm_head`) was also made trainable (`modules_to_save=["lm_head"]`) to better adapt the model to generate the required JSON output structure.

7.2.2 Hard Prompt Design

To guide the model towards the desired task and output format, a specific hard prompt was designed (defined as `UTIL_PROMPT` in `util.py`). This prompt instructs the model to act as a UI/UX reviewer, analyze the provided screenshot, and return its findings in a predefined JSON format. The JSON structure includes a boolean field `has_dark_pattern` and a list `dark_patterns`, where each entry details the `type`, `description`, and `bounding_box` of an identified dark pattern.

Prompt for Dark Pattern Detection

System Role You are a professional UI/UX reviewer specializing in detecting deceptive design elements (i.e., dark patterns).

Task Given a UI screenshot, analyze it carefully and determine whether it contains any of the following dark pattern types:

- Nagging
- Forced Action
- Urgency/Scarcity
- Misdirection

Output Specification If dark patterns exist, return a JSON object that includes the detection flag and list of identified patterns with descriptions and bounding boxes.

Example JSON output if dark patterns are found:

```
{
  "has_dark_pattern": true,
  "dark_patterns": [
    {
      "type": "Nagging",
      "description": "Repeated pop-up preventing exit",
      "bounding_box": [12, 45, 200, 150]
    }
  ]
}
```

Example JSON output if no dark pattern is found:

```
{
  "has_dark_pattern": false,
  "dark_patterns": []
}
```

Final instruction: Analyze this UI screenshot: `<Image will be provided here>`

During training data preparation (as implemented in `util.load_dataset` and the `DarkPatternFtDataset` class in `train_qwen2.ipynb`), the user input portion of each training sample incorporates this hard prompt, while the target assistant response is the corresponding ground truth JSON string.

7.3 Training Process and Hyperparameters

The training pipeline was implemented as detailed in our `train_qwen2.5.ipynb` notebook.

Data Preparation Training and validation samples, loaded via `util.load_dataset`, consist of image paths, the standardized input prompt, and the target JSON response. Within the `DarkPatternFtDataset`, images are resized (e.g., to 280x280 pixels), and the `AutoProcessor` tokenizes text and processes images into model-compatible tensors. The labels for causal language modeling are derived by cloning the input token IDs and masking the prompt portion.

Training Arguments Key hyperparameters used in `TrainingArguments` include:

- `num_train_epochs`: at least 4 for complete convergence.
- `per_device_train_batch_size`: e.g., 2.
- `gradient_accumulation_steps`: e.g., 8 (effective batch size of 16).
- `learning_rate`: e.g., $1e-4$.

- optimizer: paged_adamw_8bit.
- fp16: Enabled for mixed-precision training.
- gradient_checkpointing: Enabled to conserve memory.
- load_best_model_at_end=True with metric_for_best_model="eval_loss".

Monitoring and Output A method was used to log and plot training/validation losses and learning rates, aiding in the diagnosis of issues like overfitting (see Figure 10, Figure 11 for an example plot). Upon completion, LoRA adapters are saved, then merged with the base model, and the final fine-tuned model is stored.



Figure 10: Training and Validation Losses over Epochs of Qwen2.5-VL-7B.



Figure 11: Training and Validation Losses over Epochs of Qwen2-VL-7B.

This methodology facilitates efficient adaptation of large vision-language models to the specific nuances of dark pattern detection and structured output generation.

8 Result and Analysis

To showcase the performance enhancement of our model in discerning Dark Patterns due to our fine-tuning strategy, the variations in the model's performance across different types of Dark Patterns, and the potential correlations between model performance and various factors, we have designed a variety of statistical charts, including bar charts, radar charts, scatter plots, and confusion matrices. The following provides an explanation of our visualization results.

8.1 Model Performance Overview

We utilized bar charts (Figure 12) to illustrate the performance of Qwen2-VL and Qwen2.5-VL, both before and after fine-tuning, across four metrics of varying stringency. As can be seen, fine-tuning significantly enhanced model performance, and Qwen2.5 generally outperformed Qwen2. However, it is noteworthy that the IoU (Intersection over Union) scores exhibited anomalous behavior both before and after fine-tuning. This is because the IoU scores in this chart were calculated only for data recognized within a certain accuracy threshold. Consequently, Qwen2, prior to fine-tuning, yielded a score of 0 due to its complete inability to perform accurate recognition. Conversely, the fine-tuned Qwen2 only accurately matched a small subset of data highly similar to the training set, which resulted in its IoU score appearing artificially high.

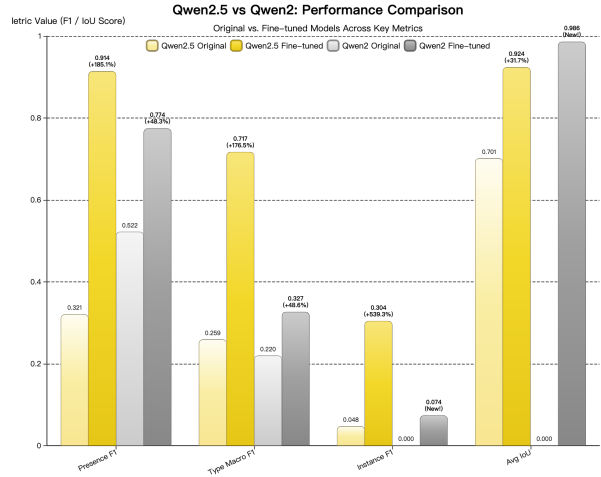


Figure 12: The adjacent bars in the figure above respectively show the pre- and post-fine-tuning performance of Qwen2.5-VL and Qwen2-VL on the metrics indicated by the x-axis.

8.2 Model Performance: Aspect Comparison

We will use radar charts (Figure 13 and 14) for a more comprehensive comparison of performance metrics for the specific models before and after fine-tuning. The evaluation metrics include precision, accuracy, and F1-Score under various conditions. For both models, an issue was observed where the 'existence precision' after fine-tuning was lower than before fine-tuning. This is likely because the training data significantly enhanced the model's generalization ability, consequently leading to a slight trade-off

in this particular precision metric.

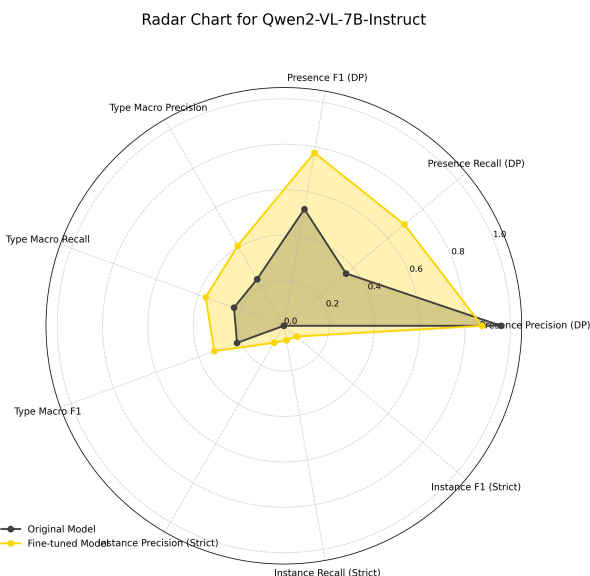


Figure 13: The figure above shows a comparison of the Qwen2-VL model’s performance across various metrics, before and after fine-tuning.

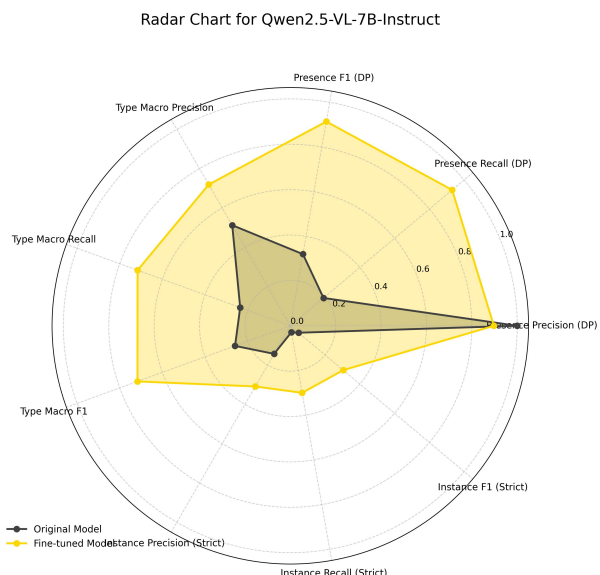


Figure 14: The figure above shows a comparison of the Qwen2.5-VL model’s performance across various metrics, before and after fine-tuning.

8.3 Detailed Exploration of Model Classification Ability

We utilized a **confusion matrix** to further investigate the model’s ability to classify different Dark Pattern types. Specifically, in this matrix, the horizontal axis represents the Dark Pattern types predicted by the model, while the vertical axis indicates the actual Dark Pattern types of the images. The values within the matrix allow us to determine whether images were correctly classified and to identify which types of images are more easily or less easily classified correctly.

For the Qwen2-VL model (**Figure 15 and 16**), it can be observed that the entries falling on the diagonal (i.e., correctly predicted images) primarily consist of images with no Dark Patterns. In contrast, the original model performed extremely poorly in predicting Dark Pattern types. After fine-tuning, however, its recognition ability showed significant improvement. Notably, the capability to distinguish types such as Nagging and Urgency increased substantially. Nevertheless, overall, the misclassification rate remains relatively high.

As for Qwen2.5-VL (**Figure 17 and 18**), its baseline classification ability was similarly unpromising, with numerous Dark Pattern images of various types being misclassified as having no Dark Patterns. However, after fine-tuning, this model demonstrated remarkable capabilities.

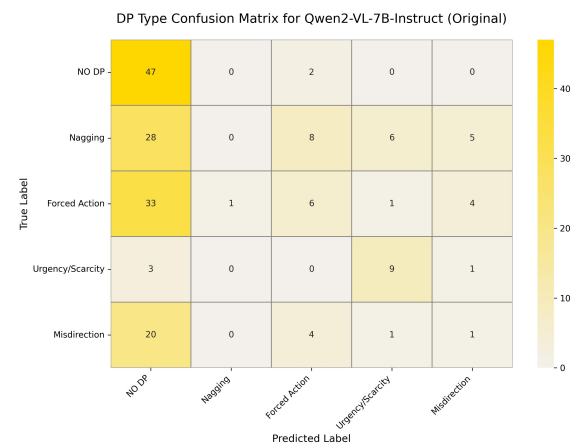


Figure 15: The figure above shows the classification ability of the original Qwen2-VL model for Dark Pattern types.

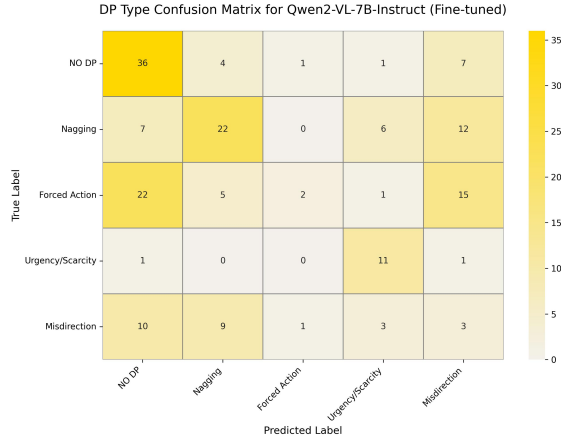


Figure 16: The figure above shows the classification ability of the fine-tuned Qwen2-VL model for Dark Pattern types

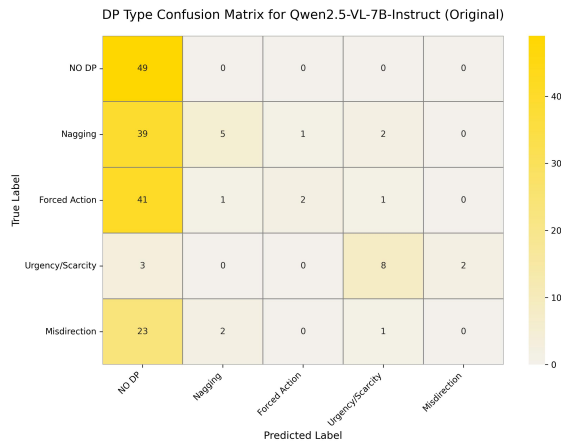


Figure 17: The figure above shows the classification ability of the original Qwen2.5-VL model for Dark Pattern types

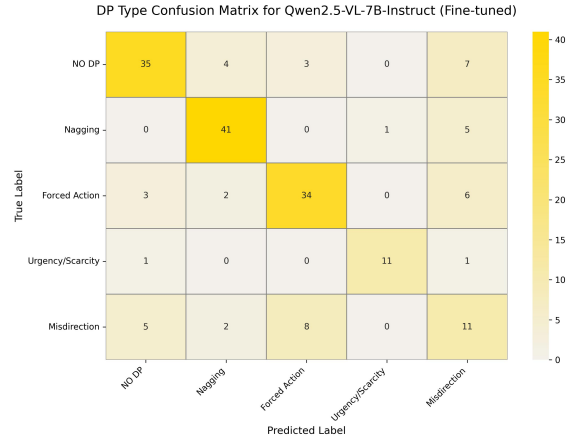


Figure 18: The figure above shows the classification ability of the fine-tuned Qwen2.5-VL model for Dark Pattern types.

The diagonal appears very dark, indicating that the vast majority of data points were correctly classified.

8.4 Detailed Distribution of IoU Scores

Rather than relying solely on a simple average score, we aim to investigate the detailed distribution of IoU scores (**Figure 19 and 20**). This is to ascertain whether low scores stem from a high concentration of genuinely low values due to the model's inherently weak recognition ability, or from a more spread-out/uniform distribution attributable to other factors. The goal is to provide distribution-informed guidance for further model optimization.

It can be observed that the fine-tuned model no longer exhibits a prominent peak in its distribution near 0, but is instead more evenly distributed. This also indicates an improvement in recognition ability.

Regarding the Qwen2.5-VL model, its performance before fine-tuning was similar to that of Qwen2, though its low-score peak was slightly shifted to the right, indicating a somewhat superior initial recognition ability. After fine-tuning, however, a new local peak even emerged in the high-score region, showcasing the outstanding performance achieved through the synergy between the fine-tuning process and the model itself.

8.5 Analysis of the Correlation Between Model Recognition Performance and Image Dimensions

In addition to straightforward data analysis, we sought to conduct further correlational studies (**Figure 21 and 22**).

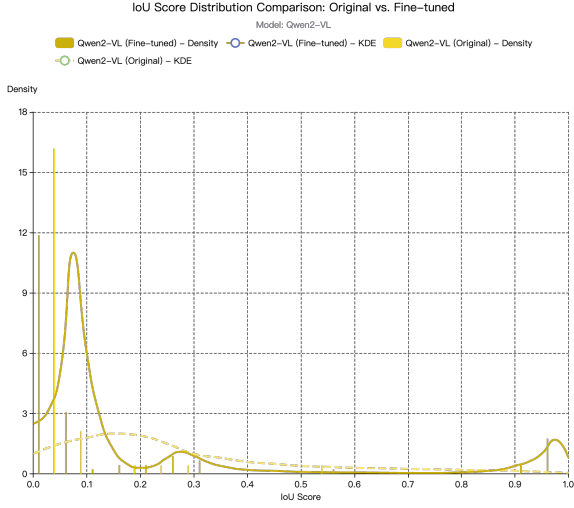


Figure 19: The figure above shows the change in the IoU score distribution for the Qwen2 model, before and after fine-tuning.

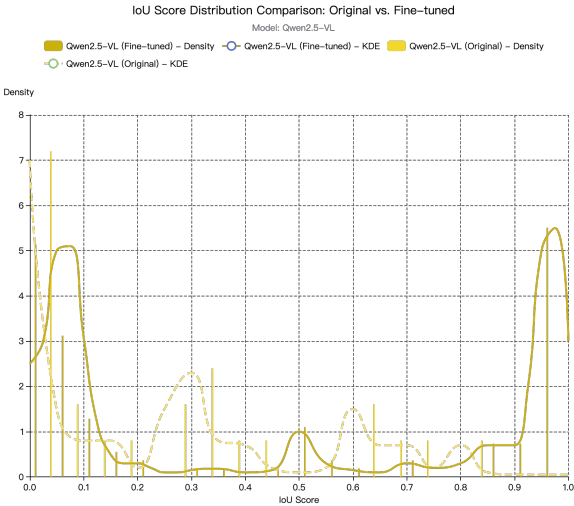


Figure 20: The figure above shows the change in the IoU score distribution for the Qwen2.5 model, before and after fine-tuning.

Noting that the dataset included image size information, we used the IoU Score as a metric to explore the connection between IoU scores and image size. The results demonstrated a degree of consistency for both Qwen2 and Qwen2.5: regardless of fine-tuning (both before and after), the models consistently achieved better performance on larger images. This is attributed to larger images being more conducive for visual models to locate Dark Pattern content. Concurrently, both fine-tuning and the use of an updated model (e.g., Qwen2.5 compared to Qwen2) led to increased IoU scores for any given image size.



Figure 21: The figure above illustrates how the IoU score distribution for the Qwen2-VL model, before and after fine-tuning, varies with image size.

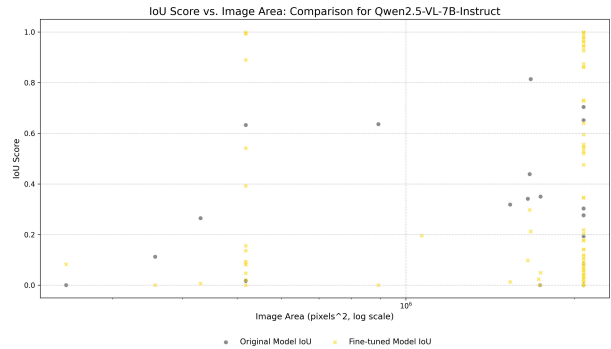


Figure 22: The figure above illustrates how the IoU score distribution for the Qwen2.5-VL model, before and after fine-tuning, varies with image size.

9 Benchmark and Comparison

Overview

We merge a combination of the AidUI and UIGuard data sets into our own data set to detect dark patterns (DP) in the user interface (UI) screenshots. Fine-tuning was performed with QLoRA for multiple language models (LLMs).

Table 2: Evaluation Metrics Across Different Models

Metric	Qwen2-VL (Orig.)	Qwen2-VL (Finetuned)	Qwen2.5-VL (Orig.)	Qwen2.5-VL (Finetuned)
DP Presence Detection (Image Level)				
Precision	0.9592	0.8750	1.0000	0.8971
Recall	0.3588	0.6947	0.1908	0.9313
F1 Score	0.5222	0.7745	0.3205	0.9139
DP Type Classification (Image Level, No BBox)				
Macro Precision	0.2366	0.4072	0.5119	0.7189
Macro Recall	0.2342	0.3672	0.2351	0.7163
Macro F1 Score	0.2199	0.3267	0.2592	0.7168
DP Instance Localization (Strict Match: Type + IoU)				
Strict Precision	0.0000	0.0865	0.1429	0.3088
Strict Recall	0.0000	0.0643	0.0286	0.3000
Strict F1 Score	0.0000	0.0738	0.0476	0.3043
Avg. IoU of TPs	0.0000	0.9862	0.7014	0.9238

Table 3: Evaluation metrics for detecting dark patterns (DP) and their classification across different models, including Qwen2-VL and Qwen2.5-VL, both in their original and finetuned versions. The metrics cover DP presence detection, DP type classification without bounding boxes, and DP instance localization with strict matching.

In this study, we focus on multiple LLMs, evaluating the Qwen2-VL and Qwen2.5-VL models in their original and fine-tuned versions, which means that there is supposed to be 4 types of models: Qwen2-VL (original): Base model without fine-tuning, Qwen2-VL (fine tuned): Fine-tuned version of Qwen2-VL with adjusted prompts, Qwen2.5-VL (original): Another base model for comparison, and Qwen2.5-VL (fine tuned): Fine-tuned version of Qwen2.5-VL.

Model Performance Analysis

1. Qwen2-VL (Original Model) The Qwen2-VL (Original) model exhibits exceptionally high precision in DP existence detection. However, its recall remains notably low, resulting in a substantial number of missed detections. In the DP type classification task, the model’s macro-average metrics are relatively weak, particularly for the Nagging category, which remains undetected. While the Urgency/Scarcity category achieves a better performance, the model is challenged in detecting Misdirection. Also, the model fails to correctly identify any instances in the strict instance localization task, which is a critical shortcoming in this aspect.

2. Qwen2-VL (Fine-tuned Model) After fine-tuning, we found a marked improvement in recall, increasing from 0.3588 to 0.6947, with a substantial rise in F1-Score, from 0.5222 to 0.7745. However, this improvement in recall comes at the cost of a slight reduction in precision. In DP type classification, the fine-tuned model performs better across all metrics, with a notable improvement in

the Nagging category. Also, instance localization is improved, with a high IoU of 0.9862, although strict precision and strict recall still remain relatively low.

3. Qwen2.5-VL (Original Model) The Qwen2.5-VL (Original) model shows almost perfect precision in DP existence detection (1.0000); however, the recall is substantially lower (0.1908), leading to a high number of missed detections. In DP type classification, the model performs moderately well, with the Urgency/Scarcity category yielding performances relatively high. Despite this, the overall performance remains limited, particularly in instance localization, where the model detects very few instances, becoming a major limitation in this aspect.

4. Qwen2.5-VL (Fine-tuned Model) We completed the Qwen2.5-VL (Fine-tuned) model, outperforming its original counterpart in all the evaluated tasks. In DP existence detection, it achieves near-perfect results, with a recall of 93 percent and an F1-Score of 91 percent. The model also performs well in DP type classification, particularly in categories such as Nagging, Forced Action, and Urgency/Scarcity, where it shows robust performance across multiple metrics. Although the model’s performance in instance localization is not flawless, the impressive IoU of 0.9238 indicates that it is highly effective at localizing the detected instances.

Analysis

In this benchmark, we conclude that Qwen2.5-VL (fine-tuned) emerged as the model with the best overall per-

formance, fitting in the existence detection of DP, classification of type DP and location of instances. Also, Qwen2-VL (fine-tuned) showed balanced improvements, especially in DP type classification, but lagged behind Qwen2.5-VL (fine tuned) in instance localization. Our fine-tuning workflow showed substantial improvements, demonstrating the capability of fine-tuned LLMs for detecting complex Dark Patterns in UI designs.

10 Conclusion

In this study, we presented a comprehensive achievement to detecting dark patterns in UI screenshots by our dataset and model fine-tuning. We enhanced a dataset with a combination of AidUI and UIGuard and explored the impact of prompt adjustments and tuning on the model's performance by employing QLoRA for model fine-tuning. Our experimental evaluation focused on various large language models (LLMs), including Qwen2-VL and Qwen2.5-VL, both in their original and fine-tuned versions.

The benchmark result reveals an insight into the effectiveness of each model variant in detecting dark patterns, emphasizing the advantages of fine-tuning in improving model accuracy. We compared performance metrics across the models with a systematic method and found significant improvements in detecting capabilities after applying fine-tuning, especially with Qwen2.5-VL.

We also contribute valuable benchmarks for LLMs we used in the experiments. We expect future researches discovering further refinements in dataset construction and fine-tuning techniques, enhancing detection across a wider spectrum of UI designs and dark patterns.

11 Acknowledgements and Suggestions

First of all, we would like to thank Professor Xu and all the tutors for bringing us such an excellent and comprehensive security course. However, regarding the course arrangement and specific experiments, we have the following suggestions:

- We suggest strengthening the connection between the lecture content and the lab exercises. The theoretical lectures are currently quite high-level, and we hope to see more technical content incorporated.
- We hope to see a better balance in the variety of topics covered by the optional labs, with more special-

ized or less mainstream choices available, such as fuzz testing included this year.

- Since it's not always easy to follow the all-English lectures, slightly slowing down the lecture pace or including some Chinese explanations on key slides would be beneficial.